



TypeScript

SOFTEX
PERNAMBUCO

 **Softex**

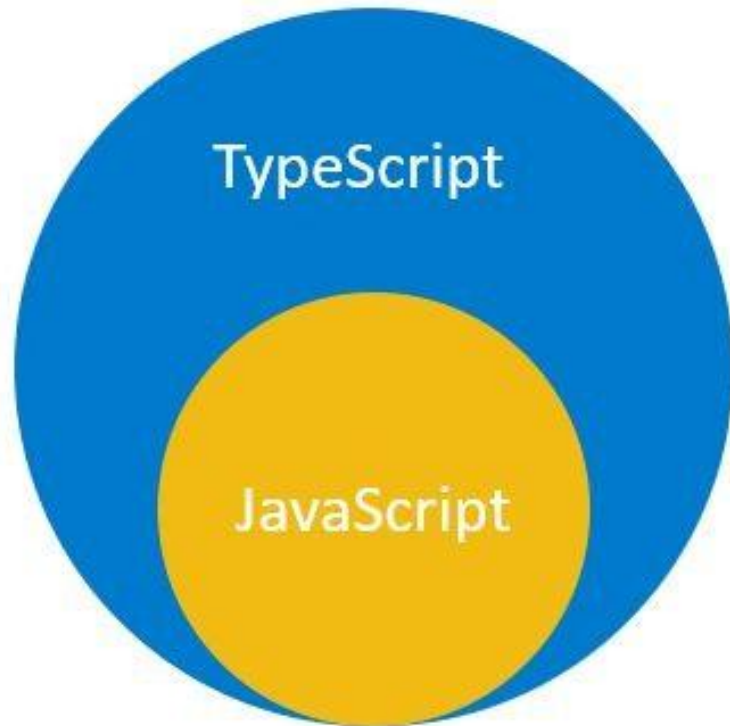
GOVERNO
FEDERAL
MINISTÉRIO DA
EDUCAÇÃO
E INOVAÇÃO

união a reed o H s-eu c x o

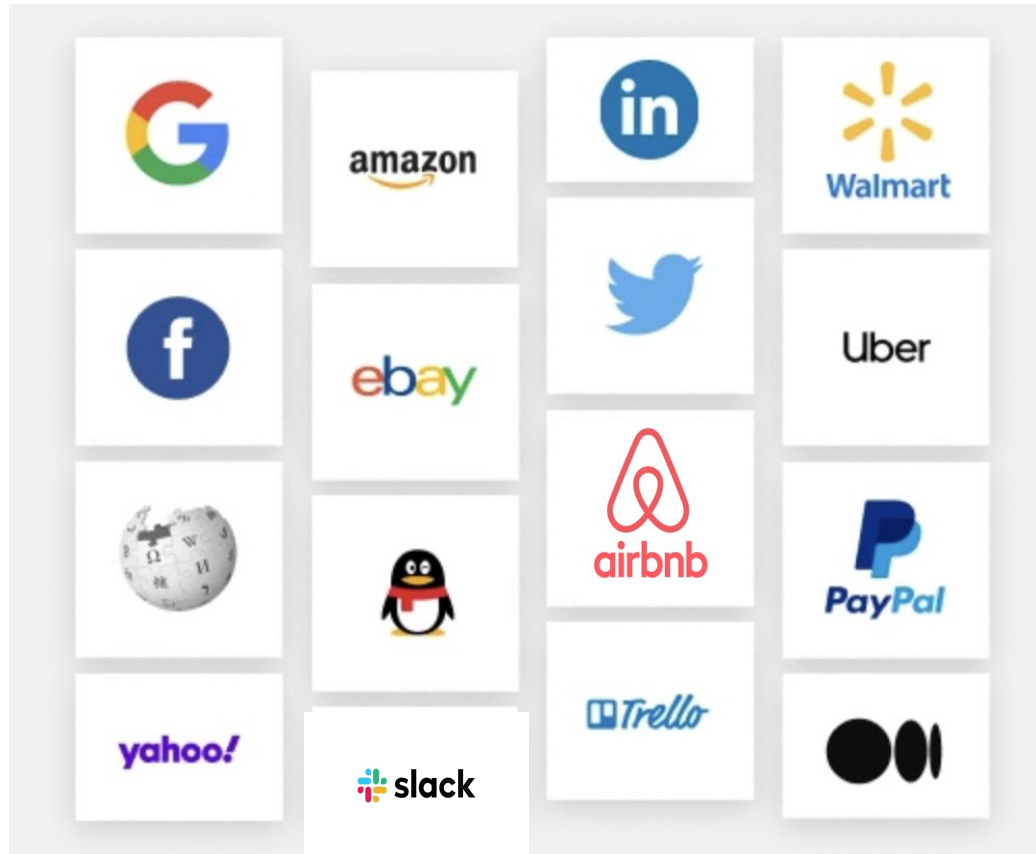
O que é TypeScript ?

- TypeScript é JavaScript com sintaxe para tipos.
- TypeScript é uma linguagem de programação fortemente tipada que se baseia em JavaScript.
- TypeScript é um superset do JavaScript.

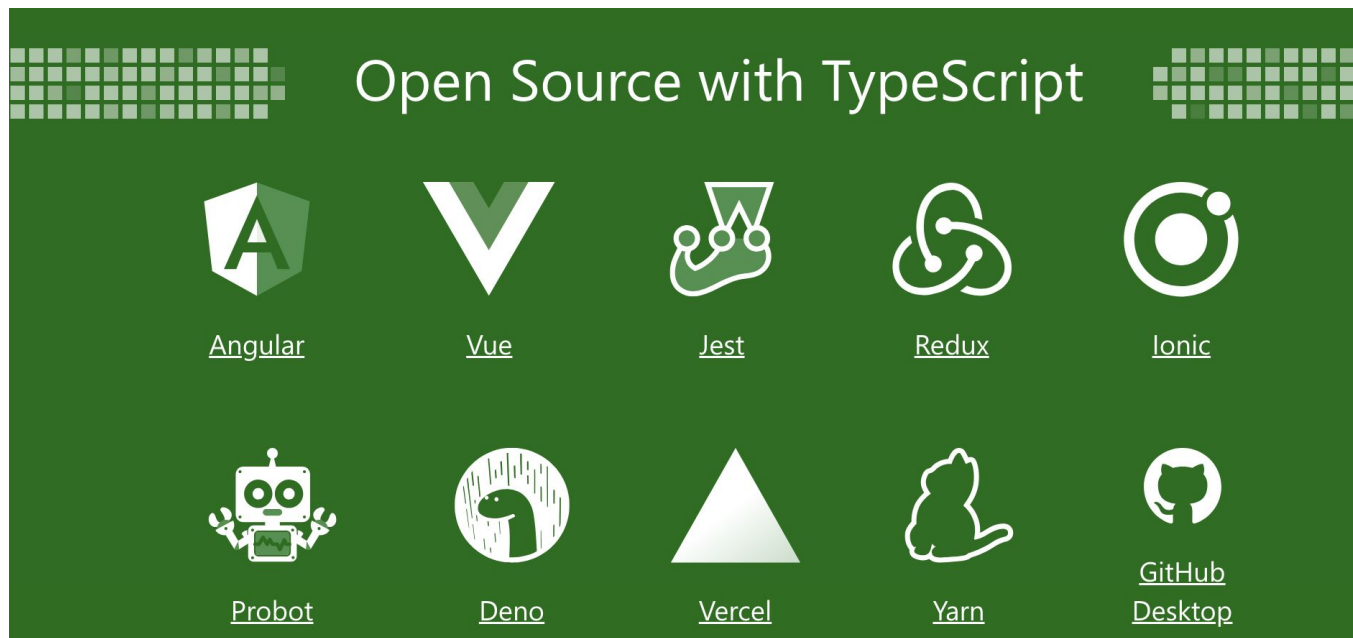
O TypeScript sendo um superconjunto sintático significa que ele compartilha a mesma sintaxe base do JavaScript, mas acrescenta algo a ela.



Quem usa TypeScript ?



Frameworks/Bibliotecas de código aberto que usam TypeScript



Por que usar TypeScript ao invés de JavaScript?



- JavaScript é uma linguagem pouco tipada
 - Pode ser difícil entender quais tipos de dados estão sendo usados.
- O TypeScript possui tipagem forte
 - Permite especificar os tipos das variáveis que estão sendo usadas
 - Tem a capacidade de relatar erros quando os tipos não correspondem.

O TypeScript usa verificação de tipo em tempo de compilação. O que significa que ele verifica se os tipos especificados correspondem **antes de** executar o código, não **durante a** execução.

JavaScript não nos ajuda na detecção prévia de erros



Sem avisos do editor no arquivo
.js
Este código quebra o ser
executado

```
→ function compact(arr) {  
    if (orr.length > 10)  
        return arr.trim(0, 10)  
    return arr  
}
```

Código JavaScript

JavaScript não nos ajuda na detecção prévia de erros



Adicionar isso ao seu código javascript irá mostrar o error

```
// @ts-check  
  
function compact(arr) {  
  if (orr.length > 10)  
    Cannot find name 'orr'.  
  return arr.trim(0, 10)  
  return arr  
}
```

o param é arr e não orr

Código JavaScript com ts-check

JavaScript não nos ajuda na detecção prévia de erros



Usando JSDoc para
obter informação do
tipo

```
// @ts-check

/** @param {any[]} arr */
function compact(arr) {
  if (arr.length > 10)
    return arr.trim(0, 10)
  return arr
}
```

Property 'trim' does not exist on type 'any[]'.

Agora TS encontrou
uma chamada inválida.
Arrays contém slide e
não trim

Função com JSDoc

JavaScript não nos ajuda na detecção prévia de erros



TypeScript adiciona
sintaxe natural para
fornecer tipos

```
function compact(arr: string[]) {  
  if (arr.length > 10)  
    return arr.slice(0, 10)  
  return arr  
}
```

Código TypeScript

TypeScript se torna JavaScript através da tecla delete



```
type Result = "pass" | "fail"

function verify(result: Result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

Código TypeScript

```
type Result = "pass" | "fail"

function verify(result: Result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

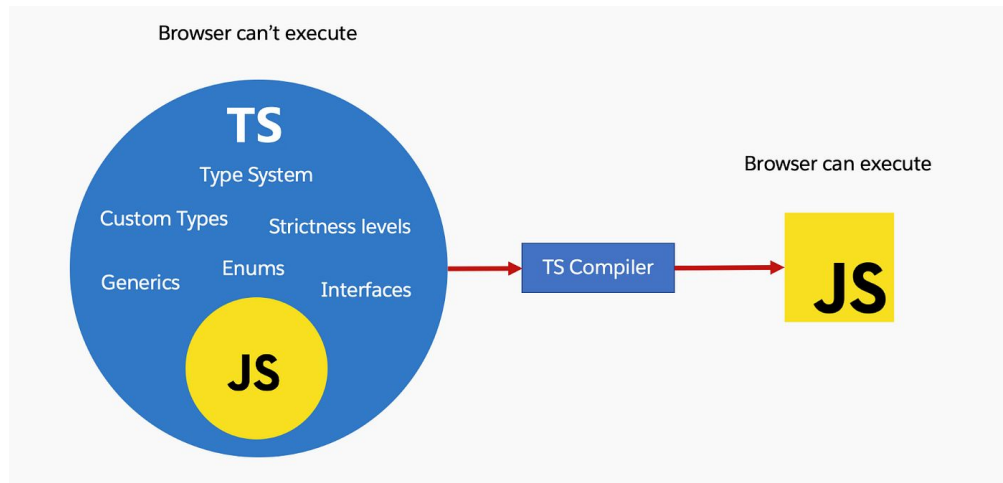
Removendo os tipos

```
function verify(result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

Código JavaScript

Como usar TypeScript?

- Uma maneira comum de usar TypeScript é usar o compilador oficial do TypeScript, que transpila o código TypeScript para JavaScript.



O TypeScript sendo convertido em JavaScript significa que ele roda em qualquer lugar onde o JavaScript roda!



Transpilação vs Compilação



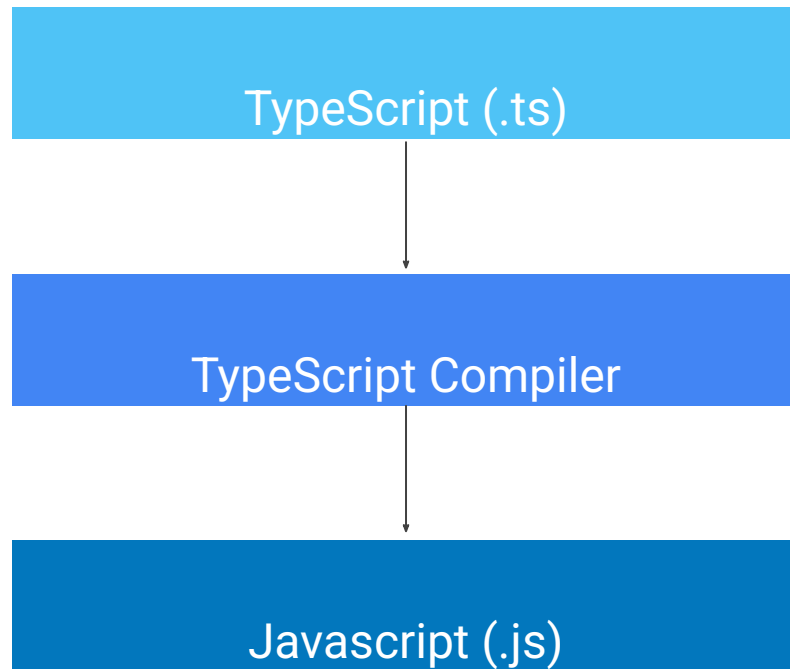
Compilação:

Alto Nível (C++) $\xrightarrow{\text{Compilação}}$ Linguagem Baixo Nível (Assembly/Bytecode)

Transpilação:

Linguagem Alto Nível (TypeScript) $\xrightarrow{\text{Transpilação}}$ Linguagem Alto Nível (JavaScript)

Transpilação



O que instalar para usar TS ?

As seguintes ferramentas você precisa instalar para começar com TypeScript:

- Node js
- TypeScript compiler

```
npm install typescript --save-dev
```

```
npx tsc
```

```
Version 4.5.5  
tsc: The TypeScript Compiler - Version 4.5.5
```



Seu primeiro programa TypeScript

Vamos criar um programa simples "Hello, World!" para verificar sua configuração do TypeScript.

1. Crie um novo arquivo `hello.ts` com o seguinte conteúdo:

hello.ts

```
function greet(name: string): string {  
    return `Hello, ${name}!`;  
}  
  
const message: string = greet("World");  
console.log(message);
```

Compilando

Exemplo

```
npx tsc hello.ts
```

Isso gerará um `hello.js` arquivo no mesmo diretório:

Exemplo

```
function greet(name) {  
  return "Hello, ".concat(name, "!");  
}  
  
const message = greet("World");  
console.log(message);
```


Executando

Exemplo

```
node hello.js
```

Você deverá ver a saída:

Exemplo

```
Hello, World!
```

O que é TypeScript ?

Resumo



Linguagem que adiciona
novas features &
vantagens ao JavaScript

JavaScript Superset

Foi construída em cima da
linguagem JavaScript

Navegadores não
rodam !!!

Benefícios do TypeScript

Resumo



- Segurança e Qualidade do Código
 - Tipagem Estática: Detecta **erros de tipo** antes da execução.
- Produtividade do Desenvolvedor
 - Melhor **IntelliSense** (Autocompletar avançado) e navegação no código.
- Escalabilidade e Manutenção (**POO** Tipado)
 - Suporte a **Interfaces** e Modificadores de Acesso (`private`, `public`).
 - Facilita a **arquitetura** de projetos grandes com POO segura.



Anotações de Tipo e Inferência

O TypeScript oferece duas maneiras de trabalhar com tipos:

1. **Digitação explícita** : você declara explicitamente o tipo de uma variável
2. **Inferência de tipo** : o TypeScript determina automaticamente o tipo com base no valor atribuído

Quando usar cada abordagem

- Use **tipos explícitos** para:
 - Parâmetros de função e tipos de retorno
 - Literais de objeto
 - Quando o valor inicial pode não ser o tipo final
- Use **inferência de tipo** para:
 - Declarações de variáveis simples com atribuição imediata
 - Quando o tipo é óbvio a partir do contexto

Anotações de Tipo Explícitas

Digitação explícita significa que você informa ao TypeScript exatamente qual tipo uma variável deve ser:

Tipos Explícitos Básicos

```
// String
greeting: string = "Hello, TypeScript!";

// Number
userCount: number = 42;

// Boolean
isLoading: boolean = true;

// Array of numbers
scores: number[] = [100, 95, 98];
```

Inferência de Tipos

O TypeScript pode determinar (inferir) automaticamente o tipo de uma variável com base em seu valor inicial:

Inferência de Tipo Básica

```
// TypeScript infers 'string'
let username = "alice";

// TypeScript infers 'number'
let score = 100;

// TypeScript infers 'boolean[]'
let flags = [true, false, true];

// TypeScript infers return type as 'number'
function add(a: number, b: number) {
  return a + b;
}
```

TypeScript Básico

Annotation e Inference

Annotation (anotação)

Com o TypeScript podemos indicar qual será o tipo de dado de uma variável através de anotações (: string).

```
ts
1  const produto: string = 'Livro';
2
3  let preco: number = 200;
4
5  const carro: {
6    marca: string;
7    portas: number;
8  } = {
9    marca: 'Audi',
10   portas: 5,
11  };
```



Inference (Inferência)

O TS consegue inferir o tipo de dado de expressões em JavaScript. Sempre que ele conseguir inferir, você não precisa fazer a anotação do dado.

```
ts
1  const produto = 'Livro';
2
3  let preco = 200;
4
5  const carro = {
6    marca: 'Audi',
7    portas: 5,
8  };
9
```



Funções

As anotações serão necessárias quando lidamos com funções.

```
1  function somar(a: number, b: number) {
2    |   return a + b;
3  }
4
5  somar("10", "10");
6
7  const nintendo = {
8    |   preco: '2000',
9  };
10
11 function transformarPreco(produto: { nome: string; preco: string }): string {
12   |   produto.preco = 'R$ ' + produto.preco;
13   |   return produto;
14 }
15
16 const produtoNovo = transformarPreco(nintendo);
17 console.log(produtoNovo);
```



Exercício 1

Conserte a função com TypeScript

```
js
1 function normalizarTexto(texto) {
2   return texto.trim().toLowerCase();
3 }
4
```

Exercício 2

Conserte as funções com TypeScript

<> index.html X

modulo2 > exercicios > <> index.html > ...

```
1 <h1>
2   Bônus de 100 - 20% do total:
3 </h1>
4 <label>Total</label>
5 <input type="number" />
6 <p></p>
7
```

TS script.ts 8 X

modulo2 > exercicios > TS script.ts > calcularGanho

```
1 const input = document.querySelector('input');
2
3 const total = localStorage.getItem('total');
4 input.value = total;
5 calcularGanho(input.value);
6
7 function calcularGanho(value) {
8   const p = document.querySelector('p');
9   p.innerText = `ganho total: ${value + 100 - value * 0.2}`;
10 }
11
12 function totalMudou() {
13   const value = Number(input.value);
14   localStorage.setItem('total', value);
15   calcularGanho(value);
16 }
17
18 input.addEventListener('keyup', totalMudou);
19
```